ALU Verification Plan

Marium Waleed Mostafa

July, 2026

ALU specification

- 4-bit ALU with two operands: `A[3:0]` and `B[3:0]`.
- Reset signal (asynchronous active low).
- 2-bit operation select `OpSel[1:0]` chooses the function:
  - `2'b00` → Addition (`Out = A + B`)
  - `2'b01` → Subtraction (`Out = A - B`)
  - `2'b00` → Bitwise AND (`Out = A & B`)
  - `2'b00` → Bitwise OR (`Out = A | B`)
- `Out[3:0]` is the 4-bit result at posedge `CLK`.
- `Carry` flag high whenever addition produces a carry-out (result > 15) or subtraction produces a borrow (result < 0). Not meaningful for AND/OR.

Signals

1. **RST**: Asynchronous active low; all output signals must be zero.
2. **4-bit inputs** `A`, `B`: Sampled at posedge `CLK`.
3. **OpSel[1:0]**: Sampled at posedge `CLK`.
   - `00`: (`Out = A + B`).
   - `01`: (`Out = A - B`).
   - `10`: (`Out = A & B`).
   - `11`: (`Out = A | B`).
4. **Out**: Checker:
   - If reset asserted → check `Out == 0`, `Carry == 0`.
   - At posedge `CLK`, according to `OpSel`.
   - if `OpSel == ADD` → check `Out == (A + B) mod 16` and `Carry == carry-out bit`.
   - if `OpSel == SUB` → check `Out == (A - B) mod 16` and `Carry == 1` when `A < B`, else `0`.
   - if `OpSel == AND` → check `Out == (A & B)` and `Carry == 0`.
   - if `OpSel == OR` → check `Out == (A | B)` and `Carry == 0`.
5. **Carry** → Checker:
   - if `OpSel == ADD` and `(A + B) > 15` → check `Carry == 1`, else `0`.
   - if `OpSel == SUB` and `A < B` → check `Carry == 1`, else `0`.

- if `OpSel == AND` → check `Carry == 0`.
- if `OpSel == OR` → check `Carry == 0`.

Functional coverage

We need to cover all output signals:

- `Out` → cover all values (0–15).
- `Out` → toggling only.
- `OpSel` → cover all 4 operations (ADD, SUB, AND, OR).
- `Carry/ADD` → cover carry-out generation (`A + B > 15`).
- `Carry/SUB` → cover borrow generation (`A < B`).
- `OpSel x A/B corner values` → cover `A == 0`, `A == 15`, `B == 0`, `B == 15`, `A == B`.
- `OpSel` → cover back-to-back operation switching (every cycle a different op).

Sequences

Sequences & Constrained Randomization

1. Reset sequence → with `RST == 0`.
2. Add sequence → with `OpSel == 2'b00`, random `A`, `B` (include `A + B > 15`, `A + B < 15`).
3. Sub sequence → with `OpSel == 2'b01`, random `A`, `B` (include `A < B`, `A == B`, `A > B`).
4. And sequence → with `OpSel == 2'b10`, random `A`, `B`.
5. Or sequence → with `OpSel == 2'b11`, random `A`, `B`.
6. Corner-case sequence → directed: `(A=0,B=0)`, `(A=15,B=15)`, `(A=15,B=0)`, `(A=0,B=15)`, `(A=B)`.

Test scenarios

- **Scenario 1**: Add sequence for random number of transactions.
  - Goal: testing addition functionality and carry-out generation.
- **Scenario 2**: Sub sequence for random number of transactions.
  - Goal: testing subtraction functionality and borrow generation.
- **Scenario 3**: And sequence → Or sequence for random number of transactions.
  - Goal: testing bitwise logic correctness (Carry = 0).
- **Scenario 4**: Add → Sub → And → Or → Add ...etc, mixing the order of `OpSel` every cycle.

  - Goal: testing correct operation switching with no stuck at faults.
- **Scenario 5**: Reset asserted mid-operation, random for at least 10 times.
  - Goal: testing asynchronous reset overrides any in-progress operation.
- **Scenario 6**: Corner-case sequence, directed, run once per corner case.
  - Goal: testing boundary values (0, 15, equal operands) across all four operations.